

Coding & Solution

Date	15 March 2026
Team ID	
Project Name	Credit Card Approval Prediction using Machine Learning
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes

2	Meaningful variable and function names are used	Yes
---	---	-----

Solution Summary

Field	Details
Repository Link / URL	https://github.com/your-team/credit-card-approval-prediction (replace with your actual repository link)
Programming Language(s)	Python
Framework(s) Used	Flask, scikit-learn, XGBoost, pandas, IBM Watson Machine Learning SDK
Key Features Implemented	Data preprocessing pipeline handling missing values, outlier treatment, and binary conversion of multi-class payment status codes; four trained classification models (Logistic Regression, Decision Tree, Random Forest, XGBoost) with comparative evaluation; best model (Random Forest) selected and serialized; Flask web application with a form for entering applicant details and instant approval/rejection prediction; IBM Watson Machine Learning deployment pipeline for cloud-hosted, scalable predictions.
Pending / Incomplete Features	Batch upload / bulk screening for the compliance officer workflow is partially implemented (single-applicant prediction only); model retraining pipeline and authentication/login for the web app are not yet implemented.
Setup / Run Instructions	1) Clone the repository and create a Python 3.8+ virtual environment. 2) Install dependencies with <code>pip install -r requirements.txt</code> . 3) Run the preprocessing and model training notebooks in /notebooks to regenerate model.pkl (optional, a pre-trained model is included). 4) Set IBM Watson ML credentials in config.py if using cloud deployment. 5) Start the Flask app with <code>python app.py</code> and open http://localhost:5000 in a browser.

Code Quality Checklist

3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The core prediction pipeline (data preprocessing, model training/evaluation, and Flask-based single-applicant prediction) is fully functional and tested end-to-end. The IBM Watson deployment has been validated with sample payloads. Next steps before final submission: implement bulk/batch screening for compliance use cases, add basic authentication to the web app, and finalize the README with environment variables and deployment steps.